

# Hermes Agent için yerel öncelikli, temalanabilir ve modlanabilir UI shell tasarım planı

## Yönetici özeti

Hermes bugün zaten birden fazla entegrasyon yüzeyi sunuyor: mevcut TUI, Node/Ink ön yüzü ile Python `tui_gateway` arka ucunu stdio üstünden newline-delimited JSON-RPC ile konuşturuyor; ayrıca ayrı bir API sunucusu `chat/completions`, `responses`, `runs`, `capabilities` ve SSE akışı veriyor; konfigürasyon, log, oturum ve profil verileri de `~/hermes` ile `HERMES_HOME` altında düzenli şekilde tutuluyor. Bu yüzden doğru strateji Hermes çekirdeğini çatallamak değil, onun önüne kendi stabil “yerel shell adaptörü”nü koymak ve UI’ı bu adaptöre bağlamak. <sup>1</sup>

Uygulanabilir en güçlü yol şudur: **ilk sürümde Python tabanlı bir adaptör + Textual tabanlı bir TUI**, ikinci aşamada ise aynı adaptör sözleşmesini kullanan **opsiyonel Ratatui** veya **Tauri+Svelte** ön yüzleri. Bunun nedeni yalın: Hermes çekirdeği Python, Textual hızlı prototipleme ve terminal/web çift hedefi veriyor, Ratatui saf TUI performansında güçlü, Tauri ise en yüksek masaüstü cilasını ve tema/ayar deneyimini sunuyor. <sup>2</sup>

Benim net önerim: **çekirdekten bağımsız bir “Hermes Local Shell” reposu** kur, Hermes’e yalnızca üç kapıdan dokun: `runs` /SSE için API sunucusu, okuma amaçlı `state.db` /log/config gözlemi, yazma ve operasyonlar için `hermes config`, `hermes sessions`, `hermes logs` gibi resmi CLI komutları. Mevcut Hermes skin sistemi ve `Hermes Mod` görsel editörü bunu destekleyen topluluk öncülü sağlıyor; ama upstream’de CLI/TUI skin parity boşlukları da belgelenmiş durumda, yani gerçek modlanabilirlik için kendi theme/layout katmanını kurman daha doğru. Güvenlik tarafında ise kabuğun varsayılanı kesinlikle local-only olmalı; Hermes API sunucusunda anahtar konfigüre edilmeden dışa açılınca ciddi RCE riski raporlandı. Hermes ayrıca yerel Windows yerine WSL2 yolunu belgeliyor; bu da platform kararlarının “UI cross-platform” ile “Hermes runtime cross-platform” olarak ayrı düşünülmesi gerektiğini gösteriyor. <sup>3</sup>

Bu raporun sonucunu tek cümleye indirirsem: **Hermes core’a dokunma; Python adaptörü standartlaştır; Textual ile hızlı MVP çıkar; tema ve layout sistemini veri odaklı tasarla; sonra aynı sözleşme üstünden ister Ratatui ister Tauri frontendi ekle.**

## Hermes entegrasyon noktaları ve minimal adaptör

Hermes’in dış UI için gerçekten işe yarayan yüzeyleri birbirinden ayırmak kritik. Burada en büyük hata, her şeyi aynı “integration layer” sanmak olur. Hermes’in bazı yüzeyleri **harici istemciye uygun**, bazıları ise yalnızca upstream iç mimarisinin parçası. <sup>4</sup>

**CLI yüzeyi** resmi ve zengin; `hermes chat`, `hermes logs`, `hermes config`, `hermes sessions`, `hermes profile`, `hermes dashboard`, `hermes acp` gibi komutlar belgelenmiş durumda. Özellikle `hermes config set` değerleri doğru dosyaya yönlendiriyor; secret’ları `.env`’ye, diğer ayarları `config.yaml`’a yazıyor. Bu yüzden shell’in konfigürasyon yazma işlemlerinde YAML’ı doğrudan mutate etmek yerine resmi CLI’ı çağırması en güvenli yaklaşım olur. <sup>5</sup>

**Mevcut TUI IPC yüzeyi** zengin ama özel. Upstream dokümanı, `hermes --tui` sürecinin Node/Ink ekran katmanını Python `tui_gateway` arka ucuna stdio JSON-RPC ile bağladığını açıkça söylüyor; `prompt.submit`, `message.delta/complete`, `tool.start/progress/complete`, `approval.request`, `session.list/resume`, `complete.slash`, `gateway.ready` gibi yüzeyler var. Bu, Hermes'in zaten "UI çekirdekten ayrılabilir" biçimde düşünüldüğünü gösteriyor; ama aynı zamanda bu katmanın **harici istemci için stabil public API olarak tasarlanmadığını** da gösteriyor. Hele v0.12.0'da TUI cold-start için ~%57 iyileştirme ve yeni davranışlar gelmiş olması, bu yüzeyin hâlâ hızlı evrildiğini düşündürüyor. Bu yüzden `tui_gateway` doğrudan ana entegrasyon sözleşmen olmasın; en fazla "upstream'e yakın mod" veya debug fallback olsun. <sup>6</sup>

**API yüzeyi** ise harici shell için asıl altın nokta. Hermes'in yerleşik OpenAI <sup>7</sup> -uyumlu HTTP sunucusu `POST /v1/chat/completions`, `POST /v1/responses`, `GET /v1/models`, `GET /v1/capabilities`, `POST /v1/runs`, `GET /v1/runs/{run_id}`, `GET /v1/runs/{run_id}/events`, `POST /v1/runs/{run_id}/stop`, `GET /health`, `GET /health/detailed` sağlıyor. Doküman özellikle `/v1/runs` hattını "streaming-friendly alternative" diye tanımlıyor; bu da dış UI için tam aradığın şey: bir run başlat, status çek, SSE ile tool ilerlemesini ve token delta'larını dinle, gerekirse stop gönder. Ayrıca `capabilities` uç noktası hangi feature'ların açık olduğunu makine-okunur biçimde veriyor; bu da versiyon bazlı hack yerine capability discovery tabanlı client yazmanı mümkün kılıyor. <sup>8</sup>

**Config ve profil yüzeyi** dış shell için çok önemli çünkü senin hedefin "VPS-first değil local-first". Hermes tüm durumu `~/.hermes/` altında tutuyor; `config.yaml`, `.env`, `auth.json`, `SOUL.md`, `sessions/`, `logs/` ve başka klasörler düzenli şekilde belgelenmiş. Profil sistemi de `HERMES_HOME` ile ayrı ayrı state dizinleri üretip her profile kendi config, session, log ve gateway durumunu veriyor. Yerel shell bunu "worlds", "profiles" veya "workspaces" gibi birinci sınıf UI nesnesi olarak göstermeli. <sup>9</sup>

**Log yüzeyi** shell için canlı observability paneli vermeye çok uygun. Hermes `agent.log`, `errors.log`, `gateway.log` dosyalarını resmi olarak belgeliyor; bunlar `RotatingFileHandler` ile döndürülüyor; redacting formatter kullanıyor; session context tag'leri içerebiliyor ve `hermes logs` komutu üzerinden follow/filter yapılabilir. Bu sayede shell tarafında hem doğrudan dosya tail'i hem de gerektiğinde CLI tabanlı filtreli okuma mümkün. Buradaki kural basit olmalı: **logları oku, ama log yazımını Hermes'e bırak.** <sup>10</sup>

**Session storage yüzeyi** doğrudan local shell için paha biçilemez. Hermes oturumları hem `~/.hermes/state.db` içinde SQLite/FTS5 ile, hem de `~/.hermes/sessions/` altında transcript dosyalarıyla takip ediyor; `state.db` WAL modunda çalışıyor ve session metadata, message history, source tag, lineage gibi bilgileri tutuyor. Bu, shell'in session picker, arama ekranı, geçmiş sohbet listesi ve istatistik panelleri için büyük fırsat. Ama burada da yazma yerine **okuma** ilkesi geçerli olmalı; `state.db` üzerinde doğrudan mutate yapmak yerine resmi Hermes çağrılarını veya gelecekte eklenecek upstream endpoint'leri kullanmak daha güvenli. <sup>11</sup>

**ACP yüzeyi** birincil hedef olmasın ama geleceğe açık dursun. Hermes ACP modunda stdio JSON-RPC sunuyor; editörler için chat, tool activity, file diff, terminal command, approval prompt ve response chunk'ları yayınlayabiliyor. Bu, "editor-native shell" veya ileride Zed / JetBrains / başka ACP istemcileriyle birleşmek istersen iyi bir ikinci kanal. Ama standalone local shell için Runs API daha doğal ve daha az semantik fazlalık içerir. <sup>12</sup>

Buradan çıkan **minimal adaptör yaklaşımı** şudur:

1. **Turn yürütme ve streaming** için Hermes API server'ı kullan. Birincil yol `POST /v1/runs` + `GET /v1/runs/{id}/events` olsun.
2. **Capability discovery** için her açılışta `GET /v1/capabilities` oku; feature flag'leri versiyon numarasından değil buradan belirle.
3. **Session browser ve local dashboard** için `state.db`, `logs/`, `config.yaml`, `~/.hermes/skins/` gibi yerleri read-only izle.
4. **Config mutation, login, profile operations, export/import** gibi yazma gerektiren işlemler için resmi CLI komutlarını sarmala.
5. **PTY embedding** modunu sadece fallback olarak düşün; bu mod en düşük bakım maliyeti verir ama en düşük tasarım özgürlüğünü sağlar.

Bu stratejinin pratik olarak doğru çalıştığını zaten Hermes'in kendi Open WebUI dökümanı da kanıtlıyor: dış bir UI, Hermes'e server-to-server bağlanıp SSE ile tool progress ve final response akışı alabiliyor. Senin shell'in farkı, bunu tarayıcıya değil yerel ve modlanabilir bir kabuğa çevirmek olacak. <sup>13</sup>

Güvenlik ve platform notu burada çok önemli. Default bind mutlaka `127.0.0.1` olmalı; `API_SERVER_KEY` her local oturumda zorunlu kabul edilmeli; mümkünse shell-adapter katmanında loopback TCP yerine Unix domain socket veya named pipe tercih edilmeli. Bunun nedeni teorik değil: Hermes API sunucusu için, anahtar yokken dışa açılma halinde prompt üzerinden terminal tool çalıştırılarak tam RCE zinciri raporlandı. Ayrıca Hermes kendi README'sinde native Windows'u desteklemeyip WSL2 öneriyor; dashboard PTY köprüsünün de POSIX PTY üzerinden çalıştığı ve native Windows'ta olmadığını upstream açıkça belirtmiş. Yani "shell tasarımı cross-platform" ile "Hermes runtime desteği cross-platform" aynı cümle değildir. <sup>14</sup>

## Yığın karşılaştırması ve önerilen yol

Aşağıdaki tablo, gerçekten uygulanabilir seçenekleri **MVP hızı, bakım riski, modlanabilirlik** ve **Hermes'e bağlanma kolaylığı** açısından özetliyor. Tablo bilinçli olarak "hangi teknoloji daha havalı" yerine "hangi yol seni en hızlı ve en az kırıma ile hedefe götürür" sorusuna cevap veriyor.

| Seçenek                         | Güçlü tarafı                                                                | Zayıf tarafı                                                                     | Hermes ile bağlama şekli                                          | En doğru kullanım                                        |
|---------------------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------|----------------------------------------------------------|
| <b>Rust + Ratatui/Crossterm</b> | Çok güçlü saf TUI performansı, tek binary hissi, keyboard-first UX          | Tema/layout motorunu çok şeyi kendin kurarsın; Python core ile süreç sınırı şart | API server + adapter üzerinden                                    | "Ben terminal-native kalacağım, Rust istiyorum" diyorsan |
| <b>Python + Textual/Rich</b>    | En hızlı geliştirme, Hermes ile dil uyumu, CSS-benzeri stil, test kolaylığı | Dağıtım biraz daha karmaşık; büyük görsel cila için masaüstü kadar rahat değil   | Doğrudan adapter veya gerektiğinde daha yakın Python entegrasyonu | <b>MVP için en iyi yol</b>                               |

| Seenek                                | Güçlü tarafı                                                                                 | Zayıf tarafı                                                                         | Hermes ile bağlama şekli               | En doğru kullanım                                        |
|----------------------------------------|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|----------------------------------------|----------------------------------------------------------|
| <b>Tauri + Svelte</b>                  | En güçlü modern masaüstü hissi, ayarlar/tema mağazası için çok uygun, görsel esneklik yüksek | Mimari karmaşıklık daha fazla; per-OS build pipeline gerekiyor                       | Local adapter + HTTP/SSE/WS            | “Terminal değil, modern desktop shell” istiyorsan        |
| <b>Warp-benzeri özel Rust renderer</b> | Teorik olarak en yüksek görsel/perf tavanı                                                   | Devasa maliyet, özel framework yükü, gereksiz erken optimizasyon                     | Tamamen özel köprü gerekir             | MVP için <b>yanlış</b> , uzun vadeli Ar-Ge için referans |
| <b>Upstream Ink TUI'yi fork etmek</b>  | En az davranış sapması                                                                       | Upstream'e çok sıkı bağlanırsın; private JSON-RPC ve Node toolchain'e mahkûm olursun | <code>tui_gateway</code> iç sözleşmesi | Genel çözüm değil, sadece upstream katkı yapmak istersen |

Bu karşılaştırmanın teknik zemini net: Ratatui modern TUI'ler için güçlü bir Rust ekosistemi sunuyor ve Crossterm saf Rust, cross-platform terminal kontrolü veriyor; Textual terminal veya tarayıcıda çalışabilen, Python tabanlı bir UI framework'ü ve Rich daha çok render/formatting katmanı; Tauri sistem webview kullandığı için küçük paket ve herhangi bir frontend ile esnek mimari vadediyor ama per-platform native toolchain istiyor. <sup>15</sup>

Benim güçlü önerim şudur: **MVP'yi Textual ile çıkar, protokolü dondur, sonra isteyenler için Ratatui veya Tauri frontendlerini aynı adapter sözleşmesi üstüne inşa et.** Bunun en büyük avantajı, seni ilk günden “Rust UI framework kararı”na kilitlememesidir. Hermes çekirdeği zaten Python; mevcut upstream TUI bile Node/Ink ile Python backend ayırımı yapıyor. Senin yapman gereken, benzer ayırımı bir adım daha genelleştirip UI-agnostik hale getirmek. <sup>16</sup>

Rust tarafını tamamen dışlamıyorum; tam tersine, **ikinci frontend olarak Ratatui çok mantıklı.** Çünkü bu kullanıcı kitlesinin bir kısmı terminal deneyimini saf, hızlı ve dependency-light ister. Ama Rust frontend'i ilk sıraya koyarsan, modlama/stil/layout tarafındaki tüm ergonomi yatırımı da ilk günden kendin yapmak zorunda kalırsın. Textual tarafında ise bu maliyeti daha hızlı ödersin ve UX kararlarını çok daha çabuk doğrularsın. <sup>17</sup>

Tauri'yi de “gereksiz webcilik” diye kenara itmemek gerekir. Senin tarif ettiğin şey yalnızca bir chat alanı değil; **tema stüdyosu, layout değiştiriciler, mod yükleyici, session browser, log inspector, world/profile yönetimi** gibi zengin paneller de var. Bu nedenle Tauri, ikinci fazda “desktop shell” olarak çok güçlü olur. Ancak birinci fazda onun maliyeti, özellikle build/distribution ve process orchestration tarafında yüksektir. <sup>18</sup>

“Warp gibi Rust framework olur mu?” sorusunun cevabı ise şu: **Warp burada çerçeve değil, referans ürün.** Resmi dökümanlarında Warp kendini Rust ile yazılmış modern terminal ve agentic development environment olarak tarif ediyor; blog yazıları da block modeli, agent/terminal ayırımı ve GPU'ya giden özel renderer mimarisini anlatıyor. Bu değerlidir, ama senin shell'ine doğrudan bağımlılık olarak alınacak

bir UI toolkit değildir. Onun kodunu veya rendering yaklaşımını kendi MVP'ne taşımaya çalışmak, gereksiz ölçüde pahalı olur. <sup>19</sup>

## Tema sistemi ve Minecraft konsepti

Hermes'in bugün zaten bir skin sistemi var: YAML dosyaları `~/ .hermes/skins/` altında tutuluyor, eksik alanlar `default` 'tan miras alıyor, renkler/spinner/branding/tool emoji/bannner ASCII art gibi alanlar destekleniyor ve user skin'ler built-in skin'lerin önüne geçebiliyor. Toplulukta `Hermes Mod` diye görsel bir skin editörü bile çıkmış durumda. Buna rağmen upstream issue'ları, CLI ile yeni TUI arasında skin parity eksikleri ve startup/session panelinin yeterince konfigüre edilememesi gibi sorunları açıkça işaret ediyor. Bu yüzden yeni shell'in tema sistemi Hermes skin'leriyle **uyumlu içe aktarma** desteği vermeli, ama kendi içinde **daha geniş kapsamlı ve UI-first** bir sistem kurmalı. <sup>20</sup>

Benim önerim: **canonical tema formatı TOML olsun**, ama **Hermes YAML import/export** desteği ilk günden bulunsun. Nedeni basit: bu shell'de yalnızca renkler değil, layout slot'ları, ikon setleri, panel isimleri, border glyph'leri, yoğunluk ayarları ve bileşen görünürlüğü gibi şeyler de değişecek. Yani burada "skin" değil, aslında **theme pack + layout pack** konuşuyoruz.

Tema sistemini üç kata ayır:

| Katman                | Ne değiştirir                                                   | Kod gerekir mi           |
|-----------------------|-----------------------------------------------------------------|--------------------------|
| <b>Theme pack</b>     | renk, ikon, border, metin etiketleri, panel görünümü            | Hayır                    |
| <b>Layout pack</b>    | sol/sağ panel, oranlar, gizli/açık bileşenler, yoğunluk modları | Hayır                    |
| <b>Widget plug-in</b> | yeni panel veya özel görsel bileşen                             | MVP'de hayır, sonra evet |

MVP'de üçüncü katmanı özellikle **ertelemeyi** öneriyorum. Çünkü executable plugin'ler local-first bir shell'de bile güvenlik, sürüm uyumluluğu ve crash isolation maliyeti getirir. İlk sürümde kullanıcıya gerçek özgürlüğü zaten theme + layout kombinasyonu verir.

Önerilen override sırası şu olsun:

1. **Built-in base theme**
2. **Seçilen theme pack**
3. **Seçilen layout pack**
4. **Profil override** ( `$HERMES_HOME/ui-shell/overrides.toml` )
5. **Workspace override** ( `.hermes-shell.toml` )
6. **Geçici runtime override** ( `: theme set` veya settings panelinden ephemeral değişiklik)

Bu sıralama, Hermes'in profil mantığıyla da doğal biçimde uyumludur; yani kullanıcının "coder" profili başka, "research" profili başka görünümle açılabilir. <sup>21</sup>

Aşağıdaki örnek, **Minecraft Overworld** teması için uygulanabilir bir temel manifest'tir:

```
[meta]
id = "minecraft-overworld"
```

```

name = "Minecraft Overworld"
version = "0.1.0"
author = "community"
extends = "default-dark"
description = "Çimen, taş ve redstone tonlarıyla block hissi veren local-
first Hermes shell teması."

[compat]
shell_api = "^1.0"
adapter_api = "^1.0"
min_hermes = "0.11.0"

[palette]
bg = "#1a1d14"
surface = "#2f3b1f"
surface_alt = "#435a2d"
panel = "#55331b"
border = "#8b5a2b"
text = "#eef6d2"
muted = "#b8c79f"
accent = "#7cb342"
accent_alt = "#c0ca33"
ok = "#66bb6a"
warn = "#ffc028"
danger = "#ff7043"
info = "#64b5f6"
status_bg = "#243118"

[borders]
style = "blocky"
horizontal = "■"
vertical = "■"
corner_tl = "■"
corner_tr = "■"
corner_bl = "■"
corner_br = "■"
separator = "▤"

[icons]
profile = "🌐"
session = "🕒"
memory = "💾"
tools = "🔧"
logs = "📖"
command = "🧱"
automation = "🔴"
agent = "👤"
approval = "❤️"

[labels]
profiles = "Dünyalar"

```

```

sessions = "Ender Chest"
memory = "Sandık"
tools = "Envanter"
composer = "Komut Bloğu"
automation = "Redstone"
logs = "Sunucu Kayıtları"
send = "Yerleştir"
stop = "Durdur"
resume = "Devam Et"

[layout]
mode = "triple-pane"
left_width = 22
right_width = 26
bottom_height = 4
show_logs_panel = true
show_memory_panel = true
show_tool_activity = true
message_style = "stacked-blocks"
tool_progress_style = "chips"

[chat]
assistant_prefix = "🧙 Hermes"
user_prefix = "🧑 Oyuncu"
tool_prefix = "🔧 Araç"
approval_prefix = "❤ Onay"

[assets]
banner = ""
███ MINECRAFT HERMES ███
██ Dünya seç, komut kaz, build üret ███
""

```

Bu dosyaya ek olarak her theme pack içinde bir `preview.txt`, `screenshot.ansi` veya küçük bir `palette.png` bulundu; mağaza/kurulum ekranında kullanıcı tema seçmeden önce ne kurduğunu görsün. Bu, mod ekosistemi için küçük ama çok fark yaratan bir UX detaydır.

Hermes'ten import için bire bir eşleme mantığı şu olabilir:

| Hermes alanı                                        | Shell alanı                        |
|-----------------------------------------------------|------------------------------------|
| <code>colors.response_border</code>                 | <code>palette.border</code>        |
| <code>colors.status_bar_bg</code>                   | <code>palette.status_bg</code>     |
| <code>branding.response_label</code>                | <code>chat.assistant_prefix</code> |
| <code>tool_emojis.*</code>                          | <code>icons.*</code>               |
| <code>banner_logo</code> , <code>banner_hero</code> | <code>assets.banner</code>         |
| <code>spinner.*</code>                              | <code>activity.spinner.*</code>    |

Bu sayede bugün elinde Hermes için yazılmış bir YAML skin varsa, kullanıcı tek tıkla onu yeni shell temasına çevirebilir. Hermes'in mevcut skin anahtarları ve topluluk mod aracı bunun için yeterli ön seriyi zaten veriyor. <sup>22</sup>

**Kurulum/marketplace UX** için ilk sürümde merkezi bir "store" şart değil. Daha iyi yol şudur:

- `shell theme list`
- `shell theme install ./my-theme`
- `shell theme install gh:owner/repo`
- `shell theme import-hermes ~/.hermes/skins/ares.yaml`
- `shell theme enable minecraft-overworld --profile coder`

İkinci fazda buna bir registry index eklenebilir; ama ben ilk sürümde tema paketlerini **veri paketi** olarak tutmanı, imza/checksum ve uyumluluk metadata'sını manifest'e koymanı öneririm. "Önce klasörden kurulum, sonra registry" sırası topluluk modlaması için çok daha doğal ilerler.

## Mimari, depo düzeni ve adaptör sözleşmesi

Önerdiğim mimari, upstream Hermes'i kendi yerinde bırakıp etrafına kontrollü bir dış katman örür. Bu yaklaşımın en büyük avantajı, bugün API server + state/log/config + profile yüzeyleriyle çalışıp, yarın gerekirse ACP veya hatta `tui_gateway` fallback'i ekleyebilmen. Upstream'in mevcut TUI/ACP ayrımı zaten bu yönde bir iç mimari çiziyor. <sup>23</sup>

Aşağıdaki akış diyagramı, önerilen katmanları gösteriyor:

```
flowchart LR
    UI["UI Frontend  
Textual MVP / Ratatui future / Tauri future"] --> AD["Local Shell Adapter  
stable contract"]
    AD --> API["Hermes API Server  
/v1/runs + SSE + capabilities"]
    AD --> CLI["Hermes CLI wrappers  
config / sessions / logs / profile"]
    AD --> FS["Read-only local state  
state.db / logs / config / skins"]
    API --> CORE["Hermes Core  
AI Agent + tools + sessions"]
    CLI --> CORE
    FS --> UI
```

Bu mimariyi repo düzeyinde şu şekilde kurmanı öneririm:

```
hermes-local-shell/
  apps/
    textual-shell/      # MVP frontend
    tauri-shell/        # opsiyonel masauustu frontend
    ratatui-shell/      # opsiyonel saf TUI frontend
  packages/
    hermes_shell_adapter/  # Python sidecar; source of truth
```



```

protocol/                # JSON Schema / OpenAPI / event contracts
theme_schema/           # theme/layout manifest validation
themes/
  default-dark/
  default-light/
  minecraft-overworld/
docs/
  architecture/
  api/
  themes/
  contribution/
  compatibility/
tests/
  contract/
  integration/
  snapshots/
  fixtures/
scripts/
  dev/
  release/

```

Burada **source of truth** açık olsun: `packages/hermes_shell_adapter`. Frontend'ler onun üstüne gelir. Bu sayede ileride “Textual’dan Ratatui’ye geçelim mi?” sorusu ürünü kilitlemez; aynı adapter protokolünü kullanan iki ayrı istemcin olabilir.

Önerilen adaptör sözleşmesini özellikle **küçük ve stabil** tut. Kendi shell API’n, Hermes’in tüm iç yapısını dışarı sızdırmamalı; yalnızca UI’n gerçekten ihtiyaç duyduğu şeyleri vermeli.

| Uç nokta / mesaj                         | Transport | Amaç                                                  | Arkadaki gerçek kaynak              |
|------------------------------------------|-----------|-------------------------------------------------------|-------------------------------------|
| <code>GET /shell/bootstrap</code>        | HTTP      | profile, theme, capabilities, models, recent sessions | API + local state                   |
| <code>GET /shell/profiles</code>         | HTTP      | profilleri listele / aktif profili gör                | CLI + local config                  |
| <code>GET /shell/sessions</code>         | HTTP      | session metadata listesi, filtreler                   | <code>state.db</code> read-only     |
| <code>GET /shell/sessions/{id}</code>    | HTTP      | session detayları / transcript özeti                  | <code>state.db</code> + transcripts |
| <code>POST /shell/runs</code>            | HTTP      | yeni run başlat                                       | Hermes <code>/v1/runs</code>        |
| <code>GET /shell/runs/{id}/events</code> | SSE       | delta, tool progress, approval, lifecycle             | Hermes runs SSE                     |
| <code>POST /shell/runs/{id}/stop</code>  | HTTP      | aktif run’ı durdur                                    | Hermes <code>/stop</code>           |
| <code>GET /shell/logs/stream</code>      | SSE/WS    | seçili log’u canlı izle                               | log files / CLI                     |

| Uç nokta / mesaj            | Transport | Amaç                                  | Arkadaki gerçek kaynak    |
|-----------------------------|-----------|---------------------------------------|---------------------------|
| GET /shell/config           | HTTP      | shell'in göstereceği normalize config | config + env              |
| PATCH /shell/config         | HTTP      | güvenli config yazımı                 | hermes config set wrapper |
| GET /shell/themes           | HTTP      | yüklü temaları bul                    | theme dirs                |
| POST /shell/themes/install  | HTTP      | tema kur / içe aktar                  | local fs                  |
| POST /shell/themes/activate | HTTP      | tema ve layout'u etkinleştir          | shell state               |

Bu API'nin tasarım ilkesi şu olmalı: **Hermes spesifik karmaşıklık adaptörde dursun, frontend'e sızmasın.** Frontend'in bilmesi gerekenler yalnızca `run_id`, `session_id`, `profile`, `theme`, `events`, `panels`, `errors` olsun.

Önerdiğim event normalizasyonu:

- `run.started`
- `assistant.delta`
- `assistant.completed`
- `tool.started`
- `tool.progress`
- `tool.completed`
- `approval.requested`
- `approval.resolved`
- `run.completed`
- `run.failed`
- `run.cancelled`
- `log.line`
- `adapter.warning`

Burada önemli detay: istemciyi savunmacı yaz. Hermeste Nisan 2026'da raporlanan bir bug, bazı başarısız run'larda SSE tarafında `run.completed` görünüp gerçek failure semantiğinin aşağı akışa doğru yansımamasına yol açabiliyordu; bu nedenle adaptör, "terminal event + içerik + hata durumu" üçlüsünü birlikte değerlendirip gerektiğinde sentetik `run.failed` üretebilmelidir. Aynı şekilde `Idempotency-Key` kullanımı da adaptörde default hale getirilmeli. <sup>24</sup>

Önerilen hata zarfı şöyle olsun:

```
{
  "error": {
    "code": "HERMES_AUTH",
    "message": "Provider kimlik bilgileri geçersiz",
    "retryable": false,
    "source": "hermes",
  }
}
```

```
    "hint": "hermes auth veya hermes model ile sağlayıcı ayarlarını doğrula"
  }
}
```

Auth modeli için önerim:

- Varsayılan: **Unix domain socket** veya **named pipe**
- Fallback: `127.0.0.1` + rastgele bearer token
- Token dosyası: `0600` izinli, launch başına rotasyonlu
- Adapter ile Hermes API arasındaki token: ayrı tut
- Browser mode veya Tauri mode varsa: session ömrü sınırlı, restore'larda yeniden üret

Aşağıdaki sözde kod, Rust frontend'in adaptör üstünden run başlatıp SSE dinlemesini gösteriyor:

```
// pseudo-code
use request::Client;
use futures_util::StreamExt;
use serde::{Deserialize, Serialize};

#[derive(Serialize)]
struct RunRequest {
    session_id: String,
    prompt: String,
    profile: String,
}

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let client = Client::new();
    let token = load_local_token()?;

    // bootstrap
    let bootstrap = client
        .get("http://127.0.0.1:39191/shell/bootstrap")
        .bearer_auth(&token)
        .send()
        .await?
        .error_for_status()?
        .text()
        .await?;
    println!("bootstrap = {}", bootstrap);

    // start run
    let run = client
        .post("http://127.0.0.1:39191/shell/runs")
        .bearer_auth(&token)
        .json(&RunRequest {
            session_id: "world-coder-main".into(),
            prompt: "src klasorunun yapisini ozetle".into(),
            profile: "coder".into(),
        })
        .send()
        .await?
        .error_for_status()?
        .text()
        .await?;
    println!("run = {}", run);
}
```

```

    })
    .send()
    .await?
    .error_for_status()?
    .json::()
    .await?;

let run_id = run["run_id"].as_str().unwrap().to_string();

// stream events
let mut stream = client
    .get(format!("http://127.0.0.1:39191/shell/runs/{}/events", run_id))
    .bearer_auth(&token)
    .send()
    .await?
    .error_for_status()?
    .bytes_stream();

let mut buffer = String::new();

while let Some(chunk) = stream.next().await {
    let bytes = chunk?;
    buffer.push_str(&String::from_utf8_lossy(&bytes));

    while let Some(idx) = buffer.find("\n\n") {
        let event_block = buffer[..idx].to_string();
        buffer = buffer[idx + 2..].to_string();

        for line in event_block.lines() {
            if let Some(data) = line.strip_prefix("data: ") {
                let value: serde_json::Value =
serde_json::from_str(data)?;
                match value["type"].as_str().unwrap_or("") {
                    "assistant.delta" => {
render_chat_delta(value["text"].as_str().unwrap_or(""));
                    }
                    "tool.started" => {
render_tool_chip(value["tool"].as_str().unwrap_or("tool"));
                    }
                    "run.failed" => {
render_error(value["message"].as_str().unwrap_or("run failed"));
                    }
                    "run.completed" => {
                        finalize_turn();
                    }
                    _ => {}
                }
            }
        }
    }
}

```

```

    }
  }
}

Ok(())
}

```

Aynı akışın Python/Textual taraftaki basit sözde kodu:

```

# pseudo-code
import asyncio
import httpx

BASE = "http://127.0.0.1:39191"
TOKEN = load_local_token()

async def start_run(session_id: str, prompt: str, profile: str) -> str:
    async with httpx.AsyncClient(timeout=None) as client:
        resp = await client.post(
            f"{BASE}/shell/runs",
            headers={"Authorization": f"Bearer {TOKEN}"},
            json={
                "session_id": session_id,
                "prompt": prompt,
                "profile": profile,
            },
        )
        resp.raise_for_status()
        return resp.json()["run_id"]

async def stream_run(run_id: str):
    async with httpx.AsyncClient(timeout=None) as client:
        async with client.stream(
            "GET",
            f"{BASE}/shell/runs/{run_id}/events",
            headers={"Authorization": f"Bearer {TOKEN}"},
        ) as resp:
            resp.raise_for_status()
            async for line in resp.aiter_lines():
                if not line.startswith("data: "):
                    continue
                event = __import__("json").loads(line[6:])
                etype = event.get("type")

                if etype == "assistant.delta":
                    append_chat(event.get("text", ""))
                elif etype == "tool.started":
                    show_tool(event.get("tool", "tool"))
                elif etype == "approval.requested":
                    open_approval_modal(event)

```

```

        elif etype == "run.failed":
            show_error(event.get("message", "run failed"))
            break
        elif etype == "run.completed":
            mark_turn_complete()
            break

async def send_message(prompt: str):
    run_id = await start_run("world-coder-main", prompt, "coder")
    await stream_run(run_id)

asyncio.run(send_message("README ve src yapisini kisaca haritala"))

```

## UX, wireframe ve akış önerileri

Buradaki hedef “existing TUI’nin biraz daha şık halı” değil; **local kullanım mantığına göre yeniden paketlenmiş bir shell**. Yani VPS-first bir agent dashboard’u değil, laptopta ve yerel projede yaşayan bir günlük çalışma kabuğu. Hermes’in profil sistemi, session storage, logs ve local terminal backend mantığı bunu zaten teknik olarak destekliyor. <sup>25</sup>

Ben burada iki görünüm öneriyorum: **varsayılan profesyonel görünüm** ve **Minecraft layout pack**. İkisi de aynı bilgi mimarisini kullanır; sadece metafor ve görsel dil değişir.

Varsayılan wireframe:

```
┌ Profiller / Oturumlar ─────────── Sohbet / Transcript
│
└─ Sağ Panel ───────────────────┘
| aktif profil | [bloklukontrol] | model / provider
mesajlar]
|
| son oturumlar | [tool progress | token / cost / usage |
chips]
| tema / layout | [approval modal | memory / session info |
anchor]
| hızlı komutlar | [diff / file preview / |
artifacts] | bg jobs / alerts |
├──────────┴──────────┬──────────┤
| Komut satırı / composer: /resume /skills /theme /logs | attach | paste |
send |
└─ Durum çubuğu: profile | cwd | transport | stream | theme | clock |
warnings ───────────┘
```

Minecraft wireframe:

The diagram illustrates the sequence of events in the game. It features a horizontal timeline with several labeled points and segments:

- Dünyalar**: The starting point on the left.
- Crafting Table**: A point reached after **Dünyalar**.
- Ender Chest**: A point reached after **Crafting Table**.
- Redstone**: A point reached after **Ender Chest**.
- profile listesi**: A point reached after **Redstone**.
- session history / saved context**: The final point on the right, reached after **profile listesi**.

Horizontal lines connect the points in sequence, indicating the flow of the game. There are also vertical lines connecting the points to the labels below the timeline.

```

ana chat alanı | cron / bg tasks |
| seed = HERMES_HOME | memory / previous builds |
tool kartları / diff blokları | watcher / alerts |
| hızlı geçiş portalları | dosya ve transcript önizleme |
slash recipes / prompt macros | status / logs |
|
| Komut Bloğu: /build /mine /inventory /resume /theme | prompt | attach |
run
└─ Hotbar: ⌘ Tools | 📁 Memory | ⚙️ Sessions | 🧱 Commands | 🔴 Automation |
📄 Logs

```

Minecraft metafor haritası:

| Minecraft ögesi          | Shell'de karşılığı                     |
|--------------------------|----------------------------------------|
| <b>Dünya / World</b>     | Hermes profile                         |
| <b>Seed</b>              | HERMES_HOME + profile state            |
| <b>Ender Chest</b>       | session archive + memory               |
| <b>Envanter</b>          | toolset görünümü                       |
| <b>Crafting Table</b>    | composer + slash komut reçeteleri      |
| <b>Command Block</b>     | hızlı prompt / macro / built-in action |
| <b>Redstone</b>          | cron, background jobs, tool automation |
| <b>Villager / Wizard</b> | alt ajanlar / delegate task            |
| <b>Hotbar</b>            | bir tuşla açılan ana paneller          |

Bu metaforun önemli tarafı şudur: sadece “yeşil kahverengi tema” yapmıyorsun; **panel isimleri, bilgi mimarisi ve etkileşim modeli** de konseptle uyumlu oluyor. Kullanıcı isterse bunu anime, cyberpunk, minimal monokrom veya ops dashboard temasına da çevirebilir; çünkü metafor görsellikten ayrıştırılmış halde layout schema’da tanımlanır.

İki UX referansını özellikle ödünç almanı öneririm. Birincisi, Warp’ın terminal ve agent modlarını ayıran, block-based navigation ve modern multiline editing fikri; ikincisi, Toad’ın “AI app store + gerçek shell entegrasyonu” hissi. İkisinin de kodunu değil, interaction pattern’lerini al. Toad ayrıca ACP tabanlı çoklu agent entegrasyonunu ve tam shell hissini pratikte gösteriyor. Ancak her ikisinin de AGPL lisanslı açık kaynak kodu olduğu için, Hermes MIT çizgisinde kalmak istiyorsan **ilham al, kopyalama yapma**. <sup>26</sup>

Hermes’in mevcut TUI yüzeylerinden ilhamla şu UX davranışlarını aynen korumalısın:

- Streaming transcript
- Tool activity chips / rows
- Approval modal
- Session picker
- Slash completion
- Long-running turn stop
- Profile-aware startup

Bunlar bugün mevcut Hermes arayüzünde zaten ayrı “surface” olarak düşünülmüş durumda; yeni shell’in değeri bunları bozmak değil, daha anlaşılır ve temalanabilir hale getirmektir. <sup>27</sup>

## MVP kapsamı ve teslim planı

MVP’yi başarıyla tanımlamanın yolu, “tam feature parity” peşinde koşmamak. İlk sürümün başarısı, Hermes’in bütün tool ekosistemini yeniden çizmekte değil; **yerel kullanıcı için net, modern, themeable, modlanabilir bir shell** vermektir.

Benim önerdiğim **MVP kapsamı** şu:

- Canlı stream alanı olan modern sohbet görünümü
- Session browser: listeleme, devam etme, başlıkla açma
- Profile/world switcher
- Model/provider göstergesi ve temel config paneli
- Tool progress satırları / chips
- Approval modal
- Log inspector
- Theme pack + layout pack sistemi
- En az bir örnek konsept tema: **Minecraft Overworld**
- Tema import: Hermes YAML skin → shell theme
- Çalışan paketleme hattı ve dokümantasyon

Aşağıdaki tablo, tek odaklı deneyimli bir geliştiriciye göre kabaca süre tahminidir. İki kişilik ekipte sürelerin yaklaşık %30–40 kısalması makul olur.

| Milestone                         | Çıktı                                                              | Efor   | Süre      |
|-----------------------------------|--------------------------------------------------------------------|--------|-----------|
| <b>Keşif ve sözleşme dondurma</b> | adapter RFC, event model, theme schema, compatibility policy       | Small  | 1 hafta   |
| <b>Adapter çekirdeği</b>          | bootstrap, runs, SSE, stop, sessions, logs, config wrapper         | Medium | 2 hafta   |
| <b>Textual MVP shell</b>          | transcript, composer, tool chips, session picker, profile switcher | Medium | 2 hafta   |
| <b>Tema motoru</b>                | TOML theme packs, layout packs, hot reload, import pipeline        | Medium | 1–2 hafta |
| <b>Minecraft tema + polish</b>    | örnek konsept tema, labels/icons/layout mapping, preview assets    | Small  | 1 hafta   |
| <b>Paketleme ve CI</b>            | pipx, binary build, release artifacts, compatibility matrix        | Medium | 1–2 hafta |
| <b>Opsiyonel Tauri shell</b>      | modern desktop frontend, settings studio, theme browser            | Large  | 3–4 hafta |
| <b>Opsiyonel Ratatui shell</b>    | pure terminal Rust frontend, same adapter contract                 | Large  | 4–6 hafta |



Buna göre pratik gerçekçi plan şu olur:

- **Hafta 1:** protokol, repo, fixtures, proof-of-concept
- **Hafta 2-3:** adapter
- **Hafta 4-5:** Textual shell
- **Hafta 6:** theme engine + Minecraft
- **Hafta 7:** test, paketleme, docs
- **Hafta 8+ :** Tauri veya Rataui ikinci frontend

MVP sonrası “nice-to-have ama bekleyebilir” listesi:

- görsel theme editor
- registry/marketplace
- UI eklenti sistemi
- embedded PTY fallback mode
- artifact paneli
- diff viewer
- split-pane file browser
- voice mode surface
- ACP bridge paneli

Buradaki en önemli ürün kararı şudur: **ilk sürümde kullanıcıya theme/layout özgürlüğü ver; widget/plugin özgürlüğünü ikinci faza bırak.** Çünkü kullanıcıların çoğu önce görünümü, etiketleri, sol/sağ panelleri ve komut akışını değiştirmek ister; arbitrary code plugin ihtiyacı çok daha sonra gelir.

## Test, paketleme, lisans ve kaynak önceliği

Test stratejisi, shell projesinin kaderini belirler. Burada üç katmanlı yaklaşım öneriyorum.

Birinci katman **protokol/contract testleri** olsun. Gerçek Hermes binary’si veya checkout’u ayağa kalsın, adapter `capabilities`, `runs`, `events`, `sessions`, `logs` sözleşmesini doğrulasın. Bu testler hermetik çalışmalı; temp `HERMES_HOME`, sahte provider config, fixture session DB ve fixture log dosyaları kullanılmalı. Hermes’in kendi katkı rehberi de testlerde environment drift’i engellemek için wrapper script kullanımını özellikle vurguluyor; aynı mantığı shell repo’sunda da sürdürmek doğru olur.

27

İkinci katman **UI testleri** olsun. Textual tarafında `pytest` + async test yaklaşımı resmi olarak belgelenmiş; Rataui tarafında ise test backend ve snapshot/golden test tarifleri mevcut. Bu yüzden hangi frontend’i yazarsan yaz, “bir prompt gönderildiğinde hangi eventler hangi panellere düşüyor?” sorusunu snapshot testlerle sabitlemek çok mümkün. 28

Üçüncü katman **fixture replay** olsun. Yani gerçek bir Hermes SSE akışını kaydedip daha sonra UI’ı bu akışla tekrar oynat. Bu, özellikle streaming, tool progress, approval ve failure semantiklerinde regresyon yakalamak için çok etkilidir. Nisan 2026’daki `run.completed` / `run.failed` örneği, neden bu tür defensive replay testlerinin gerekli olduğunu güzel gösteriyor. 29

CI tarafında karar net olmalı: **Linux ve macOS gerçek birinci sınıf platformlar**, Windows ise Hermes runtime gerçekliği nedeniyle “UI-only” veya “WSL-backed” destek olarak düşünülmeli. Tauri tarafında resmi doküman meaningful cross-compilation’ın sınırlı olduğunu ve per-platform CI/CD pipeline kullanılmasını öneriyor; bu nedenle masaüstü frontend seçersen native runner matrix’i kaçınılmaz. 30

Paketleme için önerdiğim sıralama aşağıda. Bu tablo “hepsi birden” değil, **aşamalı dağıtım stratejisi** öneriyor.

| Paketleme seçeneği                            | Ne zaman                 | Artısı                                           | Eksisi                           | Tavsiye                             |
|-----------------------------------------------|--------------------------|--------------------------------------------------|----------------------------------|-------------------------------------|
| <b>Wheel + pipx</b>                           | İlk günden               | En hızlı dağıtım, Python CLI audience için doğal | Python gerektirir                | <b>Textual MVP'nin birinci yolu</b> |
| <b>PyInstaller binary</b>                     | İlk beta sonrası         | Python kurulu olmayan makinede çalışır           | Binary/debug karmaşıklıklaştı    | Tek dosya dağıtım isteyenler için   |
| <b>Rust tar.gz/zip + cargo-dist</b>           | Ratatui frontend varsa   | Güçlü release otomasyonu, installer üretimi      | Rust build zinciri gerekir       | Rust frontend için temel yol        |
| <b>Tauri native installers</b>                | Desktop shell varsa      | En iyi son kullanıcı deneyimi                    | Per-OS build ve signing maliyeti | Desktop sürüm için doğru yol        |
| <b>Homebrew tap / bottle</b> <sup>31</sup>    | Sürüm disiplini oturunca | macOS/Linux geliştiricileri için çok doğal       | Formula bakım yükü               | İkinci faz                          |
| <b>Debian</b> <sup>32</sup> <code>.deb</code> | CLI/TUI olgunlaşınca     | Linux masaüstü ve sunucu kurulumunda rahat       | Policy/metadata disiplini ister  | İkinci faz                          |

Bu paketleme önerisinin dayanağı resmi araç ekosistemidir: Python Packaging Authority <sup>33</sup> belgeleri `pipx`’in her uygulama için izole ortam açtığını ve CLI araçlarını PATH’e güvenli biçimde taşıdığını söylüyor; PyInstaller bağımlılıklarla birlikte standalone paket üretiyor; cargo-dist tarball ve installer üretimini otomatikleştiriyor; Homebrew bottle’ları binary package olarak dağıtıyor; Tauri ise native installer/bundle hattı için platforma özgü build akışı öneriyor. <sup>34</sup>

Lisans tarafında en önemli nokta şu: Hermes MIT lisanslı. Bu yüzden shell’i **MIT veya Apache-2.0** çizgisinde tutmak mantıklı. Ancak Toad ve Warp istemcisi AGPL tabanlı açık kaynak kod yayımlıyor. Bu nedenle bu projelerden **interaction pattern** ödünç alman iyi; fakat kaynak kod, theme parser, widget implementation veya renderer parçası kopyalarsan lisans rejimin değişebilir. Açık söylemek gerekirse: **Toad/Warp’tan ilham al, ama copy-paste yapma.** <sup>35</sup>

Upstream uyumluluk ve katkı stratejisi için önerim:

- Shell ayrı repo olsun.
- Hermes’e yalnızca **genelleştirilebilir küçük PR’lar** gönder.
- İstenecek upstream katkılar şunlar olsun: daha net `capabilities`, daha sağlam run failure signaling, session/query endpoint’leri, theme parity ve public adapter-friendly yüzeyler.
- Gönderilmeyecek PR türleri: “Minecraft tab”, “benim drawer düzenim”, “benim branding label’ım”, “local-only UI opinion” gibi ürün-spesifik değişiklikler.

Bu yaklaşım upstream’i kızdırmaz, merge ihtimalini artırır ve seni uzun vadede fork bakımından kurtarır. Üstelik Hermes tarafındaki hızlı UI evrimi de bunu mantıklı kılıyor. <sup>36</sup>

Uygulayıcı ekip için önerdiğim **kaynak öncelik sırası** aşağıdaki gibidir:

| Öncelik                | Kaynak grubu                     | Neden                                                                                             |
|------------------------|----------------------------------|---------------------------------------------------------------------------------------------------|
| <b>Birinci öncelik</b> | Hermes resmi repo ve dokümanları | Gerçek entegrasyon yüzeyleri, config/session/log/api/ACP doğrusu burada                           |
| <b>İkinci öncelik</b>  | Birincil kütüphane dokümanları   | Ratatui/Crossterm, Textual/Rich, Tauri kararları için tek güvenilir zemin                         |
| <b>Üçüncü öncelik</b>  | Referans projeler ve protokoller | Toad, ACP, Warp, Helix, Lazygit gibi projeler interaction pattern ve ürün kararı için ilham verir |

Pratik okuma listesi şu sırayla ilerlemeli: Hermes'te **Configuration, Sessions, Skins & Themes, API Server, ACP, AGENTS/TUI architecture**; sonra Ratatui/Crossterm, Textual/Rich, Tauri resmi belgeleri; sonra Toad repo, ACP standardı, Warp ürün ve mimari yazıları, Helix ve Lazygit gibi örnek terminal ürünleri. Toad özellikle çoklu agent UI ve shell hissi, ACP ise standardize agent bridge, Warp ise block-based UX ve agent/terminal ayrımı, Helix ile Lazygit ise terminal-native bilgi mimarisi ve custom-command ergonomisi açısından değerlidir. <sup>37</sup>

Son öneri, uygulanacak ekibe verilecek şekilde kısa ve nettir:

#### Karar

Python adapter + Textual MVP ile başla.

#### Sınır

Hermes core'a dokunma; `runs` /SSE + read-only state + resmi CLI wrapper kullan.

#### Tema

TOML tabanlı theme/layout sistemi kur; Hermes YAML import et; Minecraft temasını örnek paket olarak çıkar.

#### Gelecek

Aynı adapter sözleşmesinin üstüne Ratatui ve Tauri frontendlere kapı bırak.

#### Lisans

MIT/Apache çizgisinde kalmak istiyorsan AGPL kod kopyalama.

#### Upstream ilişkisi

UI fikrini upstream'e taşımaya çalışma; bunun yerine küçük, adapter-friendly capability PR'ları aç.

---

<sup>1</sup> <sup>4</sup> <sup>6</sup> <sup>16</sup> <sup>23</sup> <sup>27</sup> <https://github.com/NousResearch/hermes-agent/blob/main/AGENTS.md>  
<https://github.com/NousResearch/hermes-agent/blob/main/AGENTS.md>

<sup>2</sup> <sup>17</sup> <https://textual.textualize.io/>  
<https://textual.textualize.io/>

<sup>3</sup> <sup>20</sup> <sup>22</sup> <sup>33</sup> [hermes-agent/website/docs/user-guide/features/skins.md](https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/features/skins.md) at main · NousResearch/  
hermes-agent · GitHub  
<https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/features/skins.md>

- 5 7 10 <https://github.com/nousresearch/hermes-agent/blob/main/website/docs/reference/cli-commands.md>  
<https://github.com/nousresearch/hermes-agent/blob/main/website/docs/reference/cli-commands.md>
- 8 [hermes-agent/website/docs/user-guide/features/api-server.md](https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/features/api-server.md) at main · NousResearch/hermes-agent · GitHub  
<https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/features/api-server.md>
- 9 37 [hermes-agent/website/docs/user-guide/configuration.md](https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/configuration.md) at main · NousResearch/hermes-agent · GitHub  
<https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/configuration.md>
- 11 32 <https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/sessions.md>  
<https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/sessions.md>
- 12 <https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/features/acp.md>  
<https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/features/acp.md>
- 13 [hermes-agent/website/docs/user-guide/messaging/open-webui.md](https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/messaging/open-webui.md) at main · NousResearch/hermes-agent · GitHub  
<https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/messaging/open-webui.md>
- 14 [Bug]: Unauthenticated Remote Code Execution via API Server · Issue #6439 · NousResearch/hermes-agent · GitHub  
<https://github.com/NousResearch/hermes-agent/issues/6439>
- 15 <https://ratatui.rs/>  
<https://ratatui.rs/>
- 18 <https://v2.tauri.app/start/>  
<https://v2.tauri.app/start/>
- 19 26 <https://docs.warp.dev/>  
<https://docs.warp.dev/>
- 21 25 <https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/profiles.md>  
<https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/profiles.md>
- 24 29 <https://github.com/NousResearch/hermes-agent/issues/15561>  
<https://github.com/NousResearch/hermes-agent/issues/15561>
- 28 31 <https://textual.textualize.io/guide/testing/>  
<https://textual.textualize.io/guide/testing/>
- 30 <https://tauri.app/v1/guides/building/cross-platform>  
<https://tauri.app/v1/guides/building/cross-platform>
- 34 <https://packaging.python.org/guides/installing-stand-alone-command-line-tools/>  
<https://packaging.python.org/guides/installing-stand-alone-command-line-tools/>
- 35 <https://github.com/nousresearch/hermes-agent>  
<https://github.com/nousresearch/hermes-agent>
- 36 [https://github.com/NousResearch/hermes-agent/blob/main/RELEASE\\_v0.12.0.md](https://github.com/NousResearch/hermes-agent/blob/main/RELEASE_v0.12.0.md)  
[https://github.com/NousResearch/hermes-agent/blob/main/RELEASE\\_v0.12.0.md](https://github.com/NousResearch/hermes-agent/blob/main/RELEASE_v0.12.0.md)